Listing 4.11 OpenTelemetry collector configuration, services (pipeline definitions)

```
service:
  telemetry:
    logs:
      level: debug
    metrics:
      level: detailed
  pipelines:
   metrics:
      receivers: [prometheus]
      processors: [batch]
      exporters: [logging, prometheus]
    traces:
      receivers: [otlp]
      processors: [batch]
      exporters: [logging, jaeger]
```

A good practice in dev/test is to configure the collector telemetry to emit fine-grained agent logs and metrics (available via port 8888 on the collector).

This is where the pipeline definitions starts; you can have as many as you like.

Here, we define the metrics pipeline using the Prometheus receiver in the ingress path and the Prometheus exporter for egress.

We configure the traces pipeline that uses OTLP to ingest traces from ho11y and routes them to our downstream configured Jaeger instance.

WARNING If you only enable and configure a component such as a receiver or exporter, like we did in the previous steps, but don't use them in a pipeline definition, then it is like the component doesn't exist. The collector simply ignores it, and nothing works as you would expect. This is a very common pitfall I've seen in the wild and, admittedly, run into myself a couple of times.

Also, you might have noticed that in the `exporters` part, there are two entries. The reason is simple: by including the `logging` exporter, you get to see what's going on (e.g., metrics scraped) directly on the screen, since said exporter sends signals to `stdout`.

To wrap up the configuration, let's have a quick look at how downstream Prometheus (our metrics backend destination) is configured (listing 4.8): it scrapes itself, which is a good practice, and most importantly, for our overall setup to work, it scrapes the OpenTelemetry collector via `otel-collector:9292` (this is where we told the Prometheus exporter to expose its endpoint).

Listing 4.12 Prometheus server config, scraping the OpenTelemetry collector

```
global:
  scrape_interval: 5s
  evaluation_interval: 5s
scrape_configs:
  - job_name: "prometheus"
    static_configs:
    - targets: ["localhost:9090"]
  - job_name: "otel-collector"
    static_configs:
    - targets: ["otel-collector:9292"]
```

Self-scraping the Prometheus server

Scraping the Prometheus exporter in the OpenTelemetry collector

Alright, that was a lot of YAML code to look at. Now, let's run things. Fire up one terminal session and `cd` into `ch04/otel/`, and then do

```
docker-compose -f docker-compose.yaml up
```